

DESENVOLVIMENTO WEB 2

Atividade Prática — Lista de Compras

Tema: Aplicação Web para gerenciar compras de supermercado ou lista de desejos de compras.

Base teórica: Aula 03 — Fundamentos de IHC & Boas Práticas de Desenvolvimento em JavaScript.

1. Objetivo da atividade

Construir, em **Dupla**, uma página web funcional para gerenciar uma lista de compras. A aplicação deve permitir que o usuário registre itens, acompanhe o que já foi comprado e organize sua compra de maneira clara e sem frustração.

O foco não é apenas "fazer funcionar", mas aplicar conscientemente os **princípios de IHC** e as **boas práticas de código** vistos em sala. Como discutimos, *o código também é uma interface*: ele é projetado para outro ser humano ler, manter e entender depois.

2. Tecnologias exigidas

- **HTML** — estrutura semântica da página.
- **CSS** — estilização, layout responsivo e feedback visual.
- **JavaScript** — lógica de interação (adicionar, remover, marcar, filtrar itens).
- **Um framework à sua escolha** — pode ser um framework de front-end (React, Vue, Angular, Svelte...), um framework/biblioteca de CSS (Bootstrap, Tailwind...) ou outro que julgar adequado.

REQUISITO CHAVE: A escolha do framework deve ser **justificada por escrito** (ver seção 5). Não existe framework "certo": existe a justificativa coerente entre o que o projeto precisa e o que a ferramenta oferece.

3. Requisitos funcionais (o que a aplicação deve fazer)

Obrigatórios

1. Adicionar um item à lista (nome do produto; opcionalmente quantidade e categoria).
2. Remover um item da lista.
3. Marcar/desmarcar um item como "comprado", com feedback visual claro.
4. Exibir a lista de itens de forma organizada e legível.
5. Impedir a inclusão de itens inválidos (ex.: nome vazio) — prevenção de erros.

Desejáveis (para nota extra / desafio)

- Filtrar itens (todos / pendentes / comprados).
- Contador de itens (total, comprados, restantes).
- Editar um item já cadastrado.
- Persistir os dados (ex.: localStorage) para não perder a lista ao recarregar.
- Layout responsivo (funciona bem em celular e desktop).

4. Requisitos de IHC (Parte 1 da aula)

A interface deve demonstrar, na prática, os conceitos estudados. Cada item abaixo deve estar visível e comentado na apresentação/entrega:

Affordance e significantes

- Botões e campos devem "sugerir" seu uso: um botão de adicionar parece clicável; o campo de texto convida a digitar; use ícones, sombras, sublinhados ou setas como significantes.

Feedback e visibilidade de estado

- A aplicação deve comunicar o que aconteceu: item adicionado, item marcado como comprado, erro ao tentar adicionar item vazio. O usuário nunca deve ficar em dúvida sobre o resultado da sua ação.

Prevenção de erros

- Validar entradas antes de aceitar (ex.: não permitir item em branco) e, quando fizer sentido, confirmar ações destrutivas (ex.: remover item).

Consistência

- Padrões visuais previsíveis: cores, espaçamentos, posição dos botões e comportamento uniformes em toda a tela.

Redução da carga cognitiva e usabilidade

- Tela limpa, hierarquia visual clara, poucos passos para concluir cada tarefa. Facilidade de aprendizado e de uso.

LEMBRETE DA AULA: Usabilidade = facilidade de aprendizado, eficiência, memorização, baixa taxa de erros e satisfação. Pense também na **experiência do usuário (UX)**: a aplicação é agradável de usar?

5. Requisitos de código (Parte 2 da aula)

O código será avaliado como uma interface para outro desenvolvedor. Aplique:

- **Convenções de nomenclatura:** PascalCase para classes, camelCase para funções/variáveis, UPPER_SNAKE_CASE para constantes, prefixos is/has/can para booleanos.
- **Nomes autoexplicativos** (Affordance no código): nomes de funções e variáveis devem sugerir o que fazem — ex.: adicionarItem(), marcarComoComprado(), listaDeCompras.
- **Responsabilidade única:** cada função/método faz uma coisa bem feita; evite funções longas e com muitos parâmetros.
- **Funções puras e guard clauses** sempre que possível — comportamento previsível, retorno antecipado para evitar aninhamento.
- **Prevenção de erros no código:** validação de entradas; imutabilidade (Object.freeze / const) quando o dado não deve mudar.
- **Comentários que explicam o "porquê"**, não o "o quê"; código bem nomeado já diz o que faz.
- **Formatação consistente:** indentação uniforme, espaçamento entre blocos. Uso de Prettier/ESLint é bem-vindo.

JUSTIFICATIVA DO FRAMEWORK: Inclua um arquivo **README.md** (ou um bloco de texto na entrega) explicando: qual framework você escolheu, por que ele foi adequado a este projeto, e quais princípios de IHC ou boas práticas ele ajudou a implementar.

6. Formato de entrega

1. Código-fonte organizado (HTML, CSS e JS separados ou estrutura do framework escolhido), ‘versionado’ em um repositório Git.
2. README com a justificativa do framework e instruções para executar.

7. Critérios de avaliação

Critério	Descrição	Pontos
Funcionalidade	Requisitos obrigatórios funcionando corretamente.	3,0
Aplicação de IHC	Affordance, feedback, prevenção de erros, consistência e usabilidade evidentes na interface.	3,0
Boas práticas de código	Nomenclatura, responsabilidade única, clareza, funções puras e formatação.	2,0
Escolha e justificativa do framework	Coerência entre o framework escolhido e as necessidades do projeto.	1,0
Extras / UX	Requisitos desejáveis, responsividade e cuidado com a experiência.	1,0
Total		10,0

8. Reflexão que amarra tudo

IDEIA CENTRAL: Escrever bom código é um exercício de IHC. Você está projetando duas interfaces ao mesmo tempo: a que o **usuário** vê na tela e a que outro **desenvolvedor** vê no seu código. Quanto mais usáveis, consistentes e previsíveis as duas forem, melhor o resultado — menos esforço, menos erro, mais satisfação.